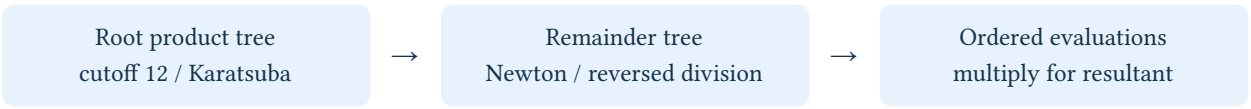
Native source polynomial engine

Prime-field companion follow-up · 2026-11-03

The Rust port preserves the source product-tree and multipoint-evaluation path, including the schoolbook cutoff of 12 coefficients, Karatsuba multiplication, Newton series inversion and reversed polynomial division.



Checked construction pipeline; no timing comparison

Checked evidence

Four bounded native test methods passed in one invocation. Three historical setup fixtures (subgroup orders 359, 479 and 683) reproduce the saved resultants and partial polynomial values. Independent schoolbook multiplication and classical long division exercise the fast paths. Horner evaluation checks order, repeated points and exceptional cases.

Source revision: d75ea6611b97b729fb562c5d01b31a71463163ba. Exact commands, compiler, binary hash and exit statuses are in receipt.json; raw output is retained in build.log and test.log.

Domain and accounting

The caller certifies prime p, bounded by half the u64 range. Products use u128. Native trimming canonicalizes coefficients. Allocation and counters are not asserted equivalent to the historical Python engine. These are unit checks; no timing comparison or full recovery cohort ran.

Integration dependency

Join the polynomial engine with native phase/division and shared-window geometry, preserving the normalized recurrence, partial tail and calibration. The original 4563-case cohort remains unchanged.

References

The original source is preserved as original-polynomial.py.txt and bound by original-inputs.sha256. See DERIVATION.md for the source alias chain and assumptions. External reference: FLINT modular-polynomial documentation. No novelty or external speed claim is made.